

보안 환경을 위한 로컬 LLM 기반 코드 어시스턴스 시스템

Abstract

본 보고서는 회사 내부 코드와 데이터를 외부 클라우드 모델에 전달할 수 없는 환경에서 사용할 수 있는 로컬 LLM 기반 코드 어시스턴스 시스템을 제안한다. 일반적인 코드 어시스턴스는 코드 작성, 코드 이해, 디버깅, 테스트 실패 분석, 리팩토링, 문서화 등 다양한 개발 작업을 지원하지만, 클라우드 API 기반으로 동작하는 경우 내부 소스 코드, 로그, 설정 파일, 실행 결과가 외부 서비스로 전송될 수 있다. 이를 피하기 위해 로컬 LLM을 사용하면 보안상 장점이 있으나, 모델 크기, context window, long-context retrieval, 양자화, 로컬 하드웨어 처리량 측면에서 성능 제약이 발생한다.

본 연구는 이러한 제약을 단순히 더 긴 context window로 해결하지 않고, 코드와 외부 데이터와 실행 중간 결과를 각각 `memory://`, `source://`, `runtime://` pointer로 다루는 구조를 제안한다. 실험 결과, 전체 repository나 전체 로그를 prompt에 직접 삽입하는 방식은 context overflow와 긴 지연 시간을 유발했다. 반면 symbol bundle, source pointer, runtime pointer를 사용하면 prompt token과 wall time을 크게 줄이면서도 정답률을 유지하거나 개선할 수 있었다. 특히 medium source pointer 실험에서 `pointer_oracle`은 `full_source` 대비 prompt token을 약 99.8%, wall time을 약 89.0% 줄였고, runtime pointer 실험에서는 `runtime_pointer_oracle`이 prompt token을 약 99.8%, wall time을 약 90.1% 줄였다.

Contents

1	Introduction	2
1.1	문제 배경	2
1.2	코드 어시스턴스의 역할	2
1.3	클라우드 모델 대비 로컬 LLM의 제약	3
1.4	성능 하락의 원인	3
2	Related Work	4
2.1	코드 생성 모델과 코드 어시스턴스	4
2.2	오픈 코드 모델과 로컬 실행	5
2.3	Context Window Overflow와 Pointer 방식	5
2.4	Self-Extend와 Long Context 확장	6
3	Proposed System	6
3.1	시스템 목표	6
3.2	구성요소별 동작 순서와 중간 산출물	7
3.2.1	symbol_index: repository 내부 코드 접근	8
3.2.2	source_pointer: disk 외부 자료 접근	9
3.2.3	runtime_pointer: 실행 중 생성된 tool output 접근	11
3.2.4	self_extend: 긴 context 직접 수용 fallback	12
3.3	개선점: 기존 시스템의 문제 해결	14
3.4	차별점: 추가된 기술과 기능	14
3.5	목적과 구현 방식	15

4	Evaluation	15
4.1	실험 환경	15
4.2	Symbol Index와 Symbol Bundle 실험	15
4.2.1	비교 방식	15
4.2.2	쉬운 질문 결과	16
4.2.3	Medium corpus 결과	16
4.2.4	Hard task 결과	17
4.3	Source Pointer 실험	18
4.3.1	비교 방식	18
4.3.2	결과 분석	18
4.4	Runtime Pointer 실험	18
4.4.1	비교 방식	18
4.4.2	결과 분석	19
4.5	Self-Extend 실험	19
4.5.1	비교 방식	19
4.5.2	결과 분석	19
4.6	종합 평가	20
5	Conclusion	20

1 Introduction

1.1 문제 배경

우리가 만들고자 하는 시스템은 Codex, Claude Code, GitHub Copilot과 같은 코드 어시스턴스이다. 코드 어시스턴스는 개발자가 자연어로 요청을 입력하면 코드베이스를 읽고, 관련 코드를 찾고, 원인을 분석하고, 필요한 경우 코드를 수정하거나 테스트를 실행하는 소프트웨어 개발 보조 시스템을 의미한다. 그러나 회사 내부 코드, 고객 데이터, 운영 로그, 보안 설정, 배포 스크립트, 테스트 결과가 외부 클라우드 서비스로 전달되는 것은 많은 조직에서 허용되지 않는다. 특히 금융, 보안, 제조, 의료, 내부 SaaS 운영 환경에서는 소스 코드 자체가 핵심 자산이므로 외부 API 기반 코드 어시스턴스를 그대로 사용하기 어렵다.

이 문제를 해결하기 위한 직접적인 대안은 로컬 환경에서 실행되는 LLM을 사용하는 것이다. 로컬 LLM은 모델 inference가 사내 장비 또는 개발자 장비에서 이루어지므로, 내부 코드와 데이터가 외부 서비스로 전송되지 않는다. 하지만 로컬 모델은 일반적으로 클라우드 frontier model보다 작고, 양자화되어 있으며, context window와 처리량이 제한된다. 따라서 단순히 “전체 repository를 prompt에 넣고 답변하게 하는 방식”은 실제 코드 어시스턴스에 적합하지 않다. 본 보고서의 핵심 문제의식은 다음과 같다.

보안을 위해 로컬 LLM을 사용하되, 로컬 모델의 context와 reasoning 한계를 고려하여 코드와 데이터 접근 방식을 pointer 중심으로 재설계해야 한다.

1.2 코드 어시스턴스의 역할

코드 어시스턴스는 단순한 자동완성보다 넓은 범위의 작업을 수행한다. 대표적인 기능은 다음과 같다.

- **코드 생성:** 요구사항을 받아 함수, 테스트, API handler, script 등을 작성한다.
- **코드 이해:** 특정 함수가 어떤 값을 반환하는지, 어떤 경로로 호출되는지, 어떤 예외가 발생하는지 설명한다.
- **디버깅:** stack trace, failing test, runtime log를 보고 원인 파일과 symbol을 찾는다.
- **코드 수정:** 버그 수정, 리팩터링, 타입 오류 수정, 설정 변경 등을 patch 형태로 만든다.

- **테스트 보조:** 테스트 실패 원인을 분석하고, 필요한 테스트를 추가하거나 기존 테스트를 갱신한다.
- **문서화:** 코드 흐름, API 사용법, 내부 convention을 문서로 정리한다.

예를 들어 개발자가 “결제 실패 로그에서 원인을 찾아 관련 코드를 수정해줘”라고 요청했다고 하자. 일반적인 코드 어시스턴스는 테스트나 로그를 읽고, 로그에 나온 request id 또는 error message를 검색하고, 관련 payment 함수와 test를 찾아서 수정안을 만든다. 이때 필요한 정보는 전체 repository나 전체 로그가 아니라 실패 line 주변, 관련 함수, helper 함수, 설정 값, 테스트 assertion이다. 따라서 코드 어시스턴스의 품질은 LLM 자체 성능뿐 아니라 “필요한 context를 얼마나 작고 정확하게 구성하는가”에 크게 좌우된다.

1.3 클라우드 모델 대비 로컬 LLM의 제약

로컬 LLM을 사용하는 경우 다음 제약이 발생한다.

- **모델 크기:** 로컬 장비에서 현실적으로 실행 가능한 모델은 7B, 14B, 32B급인 경우가 많다. 이는 클라우드 frontier model보다 parameter 수, 학습 데이터, post-training, tool-use 튜닝 면에서 작을 가능성이 높다.
- **양자화:** 로컬 실행을 위해 GGUF Q4, Q5 등으로 양자화하면 메모리 사용량은 줄지만 일부 task에서 reasoning 품질 또는 출력 안정성이 떨어질 수 있다.
- **Context window:** 전체 repository, build log, JSON snapshot, 테스트 출력은 수십만 token까지 커질 수 있다. 로컬 모델의 context size가 32k 또는 65k여도 실제 코드 어시스턴스에는 부족할 수 있다.
- **처리량과 지연 시간:** 로컬 GPU 또는 Apple Silicon unified memory 환경에서는 prompt eval 시간이 prompt token 수에 크게 의존한다. 긴 prompt를 넣으면 답변 전 대기 시간이 수십 초에서 수분으로 증가한다.
- **Long-context retrieval:** 긴 입력을 수용하는 것과 긴 입력 안에서 필요한 symbol 또는 값을 정확히 회수하는 것은 다른 문제다.

실제 논문과 기술 보고서에서도 이러한 차이를 정량적으로 확인할 수 있다. Qwen2.5-Coder Technical Report는 Qwen2.5-Coder-7B-Instruct가 HumanEval 88.4%, MBPP 83.5%, BigCodeBench Full 41.0%, Aider Pass@1 55.6%를 기록했다고 보고한다. 같은 표에서 GPT-4o-2024-08-06은 Aider Pass@1 56.8%, Claude-3.5-Sonnet-20241022는 71.4%, o1-preview는 69.9%를 기록한다. 즉 HumanEval처럼 짧은 함수 생성 문제에서는 최신 7B 코드 모델이 클라우드 모델에 근접할 수 있지만, code editing benchmark에서는 Claude-3.5-Sonnet 대비 15.8 percentage point 낮다. 또한 Self-Extend 논문의 L-Eval 표에서는 GPT-4-32k 평균이 73.11, Claude1.3-100k 평균이 65.97인 반면, Llama2-7B-chat 4k는 33.12, SelfExtend-Llama2-7B-chat 16k는 35.99로 보고된다. CodeU 항목에서도 GPT-4-32k는 25.55인 반면 Llama2-7B-chat은 1.11이다. 이는 로컬 7B급 모델에서 long-context와 코드 reasoning이 동시에 필요한 경우 성능 저하가 크게 나타날 수 있음을 보여준다.

본 연구의 자체 실험에서도 같은 문제가 관찰되었다. medium symbol 실험에서 전체 repository를 넣는 full_repo 방식은 65k context에서 실패했고, 131k context로 늘려도 한 질문 처리에 612.869초가 걸렸다. Self-Extend 실험에서는 group attention을 적용해 n_ctx를 약 8k 또는 16k로 늘릴 수 있었지만, passkey retrieval과 코드 어시스턴스 benchmark는 모두 실패했다. 따라서 긴 context 자체를 늘리는 방식만으로는 보안형 로컬 코드 어시스턴스를 안정적으로 만들기 어렵다.

1.4 성능 하락의 원인

로컬 LLM의 성능 하락은 하나의 원인으로 설명되지 않는다. 본 연구에서는 다음 네 가지 원인이 특히 중요하다고 본다.

1. **모델 capacity 차이:** 작은 모델은 복잡한 cross-file reasoning, tool-call planning, code edit formatting에서 클라우드 frontier model보다 취약하다.
2. **Context length와 positional out-of-distribution:** Transformer 계열 LLM은 token의 내용뿐 아니라 token 사이의 상대적 거리도 attention 계산에 사용한다. 문제는 모델이 학습 중 주로 경험한 길이를 넘어서는 prompt를 받으면, 이전에 거의 보지 못한 “먼 거리의 token 관계”를 처리해야 한다는 점이다. 예를 들어 4k context로 학습된 모델에 6k, 8k 입력을 넣으면 모델은 입력을 물리적으로 받을 수 있더라도, 앞부분의 함수와 뒷부분의 질문이 어떤 관계인지 안정적으로 연결하지 못할 수 있다. 이 현상을 positional out-of-distribution이라고 볼 수 있다. 코드 어시스턴스에서는 이 문제가 “관련 함수가 prompt 안에 있는데도 찾지 못함”, “비슷한 이름의 다른 함수와 혼동함”, “앞쪽에 나온 상수 정의를 뒤쪽 계산에 반영하지 못함”으로 나타난다. Self-Extend 논문 Table 1에서 Llama-2-7B-chat은 4096 token에서 PPL 9.181이지만 6144 token 이상에서는 $> 10^3$ 으로 증가한다. 이는 단순히 prompt가 길어질수록 느려지는 문제가 아니라, 모델이 긴 입력 내부의 위치 관계를 신뢰성 있게 해석하지 못할 수 있음을 의미한다.
3. **긴 context 내부의 정보 희석:** prompt가 길어지면 관련 코드가 context 안에 있더라도 distractor 함수, 유사 symbol, noise log 때문에 모델이 잘못된 위치를 참조할 수 있다. source pointer 실험의 full_source JSON task가 context 안에 들어갔음에도 distractor 값을 답한 것이 그 예다.
4. **추론과 실행의 혼합:** 코드 어시스턴스는 두 종류의 일을 동시에 해야 한다. 첫째는 “어느 코드가 필요한가”를 찾는 retrieval 문제이고, 둘째는 찾은 코드를 실제로 실행했을 때 어떤 값이 나오는지 계산하는 execution 문제이다. LLM은 Python interpreter가 아니라 다음 token을 생성하는 모델이므로, strip(), replace(), lower()를 순서대로 적용하거나 $10000 * 0.0125$ 를 정확히 계산하는 일을 항상 안정적으로 수행하지 못한다. 따라서 올바른 helper 함수와 상수를 모두 찾아 prompt에 넣어도, 모델이 그 코드를 머릿속으로 실행하는 과정에서 문자열 치환 순서나 산술 결과를 틀릴 수 있다. 본 연구의 symbol bundle 2차 실험에서도 retrieval은 성공했지만 North/East Plan의 slash를 잘못 바꾸거나 late fee를 125가 아닌 150으로 계산하는 오답이 발생했다. 즉 문제는 “정보를 못 찾은 것”이 아니라 “찾은 코드를 정확히 실행하지 못한 것”이다.

따라서 본 연구의 접근은 “로컬 모델을 더 길게 만든다”가 아니라 “LLM에게 들어가는 context를 작고 검증 가능하게 만든다”이다.

2 Related Work

2.1 코드 생성 모델과 코드 어시스턴스

Codex 논문인 “Evaluating Large Language Models Trained on Code”는 HumanEval을 통해 code generation을 기능적 정확성으로 평가했다. 이 논문에서 Codex는 HumanEval pass@1 28.8%를 기록했고, GPT-3는 0%, GPT-J는 11.4%를 기록했다. 또한 100개 sample을 생성해 선택하는 방식에서는 70.2%까지 올라갈 수 있음을 보였다. 이는 코드 어시스턴스에서 단일 답변보다 반복 생성, 검증, 테스트 실행이 중요하다는 점을 보여준다.

Code Llama 논문은 Llama 2 기반 code model을 7B, 13B, 34B, 70B 크기로 공개하고, HumanEval과 MBPP에서 open model의 성능을 크게 끌어올렸다. 논문 초록은 Code Llama가 HumanEval 최대 67%, MBPP 최대 65%를 달성한다고 보고한다. 이 결과는 open-weight 모델도 코드 생성에서는 실용적인 수준에 도달할 수 있음을 보여준다.

하지만 코드 생성 benchmark와 실제 software engineering issue 해결은 문제 구조가 다르다. HumanEval이나 MBPP는 보통 함수 signature, 요구사항, 짧은 단위 테스트가 한 prompt 안에 주어지고, 모델은 하나의 함수 본문을 생성하면 된다. 반면 실제 issue 해결에서는 먼저 어떤 파일을 봐야 하는지 찾아야 하고, 여러 함수와 class의 호출 관계를 따라가야 하며, test fixture, configuration, 기존 convention, error log, dependency version까지 함께 고려해야 한다. 또한 정답은 독립적인 함수 하나가 아니라 기존 코드베이스에 들어가는 최소 patch여야 하므로, 수정하지

말아야 할 public API나 다른 테스트와의 상호작용도 신경 써야 한다. 따라서 코드 생성 benchmark가 높다는 것은 “짧고 잘 정의된 함수 작성” 능력을 보여주지만, 실제 코드 어시스턴스에 필요한 “관련 context 검색, 원인 진단, 안전한 수정, 테스트 기반 검증” 능력을 그대로 보장하지 않는다.

SWE-bench는 실제 GitHub issue와 pull request를 기반으로 모델이 codebase를 수정해 issue를 해결할 수 있는지 평가한다. SWE-bench 논문은 issue 해결이 여러 함수, class, file을 동시에 이해해야 하고, 실행 환경과 긴 context 처리가 필요하다고 설명한다. 해당 논문에서 가장 좋은 Claude 2도 1.96%만 해결했다. 이는 “코드를 한 함수 단위로 생성하는 능력”과 “실제 repository에서 문제를 찾아 수정하는 능력” 사이에 큰 차이가 있음을 보여준다.

2.2 오픈 코드 모델과 로컬 실행

Qwen2.5-Coder Technical Report는 0.5B부터 32B까지 다양한 크기의 code-specific model을 제시한다. 특히 Qwen2.5-Coder-7B-Instruct는 HumanEval 88.4%, MBPP 83.5%, Aider Pass@1 55.6%를 기록해 7B급 모델 중 강한 성능을 보인다. 그러나 같은 보고서의 Aider 결과에서 Claude-3.5-Sonnet-20241022는 Pass@1 71.4%를 기록하므로, 실제 code editing 성능에서는 여전히 cloud model과 차이가 있다.

Qwen2.5-Coder는 file-level pretraining 이후 repo-level pretraining을 수행하고, context length를 8,192에서 32,768로 확장하며, YaRN을 통해 131,072 token까지 다룰 수 있도록 설계한다. 이는 code assistant가 repository-level context를 필요로 한다는 점을 반영한다. 본 연구는 이 방향과 유사하게 repository-level 정보를 다루지만, 전체 repo를 raw context로 넣기보다는 symbol index와 pointer resolver를 통해 필요한 부분만 선택한다.

2.3 Context Window Overflow와 Pointer 방식

“Solving Context Window Overflow in AI Agents”가 다루는 핵심 문제는 agent가 tool을 사용할 때 발생한다. 일반적인 tool-calling workflow에서는 tool의 반환값이 그대로 다음 LLM message에 삽입된다. 반환값이 짧은 문자열이면 문제가 없지만, PDF 전체 텍스트, 긴 로그, 큰 JSON, 대규모 행렬처럼 반환값이 매우 크면 두 가지 문제가 생긴다. 첫째, 반환값 자체가 context window보다 커서 다음 LLM 호출이 실패한다. 둘째, context window 안에 들어가더라도 대부분의 token이 raw data를 담는 데 쓰이므로, 모델이 실제로 필요한 정보에 attention을 집중하기 어렵고 prompt evaluation 비용도 증가한다.

단순한 truncation이나 summarization은 이 문제를 완전히 해결하지 못한다. Truncation은 뒤쪽 또는 앞쪽의 원본 정보를 버리므로 필요한 값이 잘릴 수 있다. Summarization은 token 수를 줄이지만 원본과 동일한 tool input을 보존하지 못한다. 예를 들어 어떤 tool이 앞 단계 tool의 원본 JSON 전체나 3D matrix 전체를 입력으로 받아야 한다면, 요약문은 다음 tool의 올바른 입력이 될 수 없다. 즉 agent workflow에서는 “LLM이 읽기 위한 정보”와 “다음 tool이 처리해야 하는 원본 값”을 구분해야 한다.

해당 논문은 이 구분을 위해 mirrored tool 구조를 제안한다. Mirrored tool은 원래 tool의 앞뒤에 wrapper를 둔다. Tool 출력이 작으면 기존처럼 반환하지만, 출력이 크면 runtime memory에 저장하고 LLM에게는 짧은 memory pointer와 접근 방법만 돌려준다. 이후 다른 tool이 그 pointer를 인자로 받으면 wrapper가 pointer를 원본 값으로 resolve한 뒤 원래 tool에 전달한다. LLM은 거대한 raw data를 직접 보지 않고도 “이 값을 다음 tool에 넘겨라”라는 계획을 세울 수 있고, tool은 원본 데이터를 손실 없이 사용할 수 있다.

논문 실험은 이 설계가 단순한 아이디어가 아니라 실제 context overflow를 해결한다는 점을 보여준다. Materials Science workflow에서는 $128 \times 128 \times 128$ 크기의 electronic grid, 즉 2,097,152개의 float32 값을 생성한다. Conventional workflow는 이 값을 LLM context에 넣으려다 실패하지만, pointer 방식은 grid를 memory에 저장하고 pointer만 전달하여 전체 pipeline을 완료한다. 또한 context에 들어갈 수 있는 크기의 SDS ingredient extraction 실험에서도 기존 workflow는 평균 6,411.36 token과 43.05초를 사용한 반면, pointer 방식은 841.66 token과 11.05초만 사용했다. 이는 약 7배 token 절감과 약 3.9배 시간 단축이다. 따라서 pointer 방식은 overflow가 발생하는 extreme case뿐 아니라, context에 들어갈 수는 있지만 비효율적인 중간 크기 데이터에도 효과가 있다.

본 연구의 source pointer와 runtime pointer 실험은 이 아이디어를 코드 어시스턴스에 맞게 확장한다. 코드 어시스턴스에서 큰 데이터는 tool output 만이 아니다. 이미 disk에 존재하는 운영 로그, markdown runbook, JSON snapshot도 크고, repository 내부 코드도 전체를 넣으면 과도하게 크다. 따라서 본 연구는 실행 중 생성된 값은 runtime://, disk source는 source://, 코드 symbol은 memory://symbol/...로 분리한다. 이 분리는 데이터의 lifecycle과 접근 방식이 다르기 때문에 필요하다. Runtime 값은 session 단위로 저장과 삭제가 필요하고, source file은 manifest와 파일 권한 검사가 필요하며, code symbol은 AST index와 line range provenance가 필요하다. 세 pointer를 구분하면 각 데이터 유형에 맞는 resolver와 extractor를 붙일 수 있고, LLM에게는 모든 원본을 노출하지 않고도 필요한 증거만 제공할 수 있다.

2.4 Self-Extend와 Long Context 확장

Self-Extend 논문은 fine-tuning 없이 inference 단계에서 group-level attention과 neighbor-level attention을 구성해 context window를 확장하는 방법을 제안한다. 논문 Table 1에서 Llama-2-7B-chat은 4096 token 이후 PPL이 $> 10^3$ 으로 폭증하지만, SelfExtend-Llama-2-7B-chat은 16384 token에서도 PPL 약 9 수준을 유지한다. 또한 Mistral passkey retrieval에서는 4k부터 24k까지 100% retrieval accuracy를 보고한다.

하지만 Self-Extend 논문 자체도 낮은 PPL이 실제 long-context 이해를 보장하지 않는다고 지적한다. 본 연구의 재현 실험에서도 이 점이 중요하게 나타났다. 로컬 llama.cpp 환경에서 Self-Extend는 context 수용량은 늘렸지만 passkey retrieval과 code assistance retrieval에는 실패했다. 따라서 본 연구에서는 Self-Extend를 기본 retrieval 방식으로 사용하지 않고, pointer와 symbol retrieval로 context를 줄인 뒤에도 부족한 경우에만 실험적 fallback으로 간주한다.

3 Proposed System

3.1 시스템 목표

제안 시스템의 목적은 보안이 중요한 내부 개발 환경에서 동작하는 로컬 코드 어시스턴스를 만드는 것이다. 핵심 목표는 다음과 같다.

- 내부 코드, 로그, JSON snapshot, 테스트 결과를 외부 클라우드 모델로 전송하지 않는다.
- 로컬 LLM의 제한된 context window 안에서 필요한 정보만 전달한다.
- 전체 repository나 전체 로그를 prompt에 넣는 방식을 기본값으로 사용하지 않는다.
- 코드 어시스턴스의 작업을 retrieval, extraction, deterministic execution, validation 단계로 분리해 각 단계의 입력과 출력을 검증 가능하게 만든다.

이 단계들을 분리해야 하는 이유는 각 단계의 실패 양상이 서로 다르기 때문이다. Retrieval은 “어떤 파일과 symbol이 관련 있는가”를 고르는 문제이다. 여기서 실패하면 LLM은 애초에 잘못된 근거를 보게 된다. Extraction은 “선택된 로그, 문서, JSON, 코드에서 어느 부분만 잘라낼 것인가”의 문제이다. 이 단계는 데이터 형식별 규칙이 중요하므로 log는 grep, document는 section heading, JSON은 path lookup처럼 deterministic tool로 처리하는 편이 안정적이다. Deterministic execution은 “찾은 코드나 값으로 실제 결과를 계산하는 문제”이다. 구체적인 입력값이 있는 문자열 변환, 산술, JSON path 평가는 LLM에게 추측하게 하기보다 interpreter나 evaluator가 계산해야 오차가 줄어든다. Validation은 LLM이나 selector가 만든 pointer, tool name, JSON path, patch가 schema와 allowlist를 만족하는지 확인하는 단계이다. 이 검사를 분리하면 잘못된 pointer나 잘못된 tool call이 다음 단계로 전달되기 전에 차단된다.

따라서 분리는 단순한 설계 취향이 아니라 오류 전파를 줄이기 위한 구조이다. 모든 일을 하나의 LLM prompt에 맡기면 “관련 파일 선택”, “필요 부분 추출”, “코드 실행 결과 계산”, “출력 형식 검증”이 한 번에 섞이고, 어느 부분에서 틀렸는지 알기 어렵다. 반대로 단계별로 나누면 각 단계는 작은 입력과 명확한 출력 계약을 갖는다. 실패한 단계만 재시도하거나 fallback을 적용할 수 있고,

deterministic tool로 처리 가능한 부분은 LLM의 확률적 생성에 맡기지 않아도 된다. 이 때문에 같은 로컬 LLM을 쓰더라도 최종 답변과 code patch의 신뢰성이 높아진다.

이를 위해 시스템은 세 종류의 pointer를 사용한다.

Pointer type	대상	예시
memory://symbol/...	repository 내부 코드 symbol	memory://symbol/services/payment.py::authorize_payment
source://...	disk에 존재하는 외부 자료	source://log/logs/checkout_gateway.log
runtime://...	tool 실행 중 생성된 큰 결과	runtime://log/run_tests/0001

3.2 구성요소별 동작 순서와 중간 산출물

제안 시스템은 네 개의 핵심 항목으로 구성된다. `symbol_index`는 repository 내부 코드 구조를 다루는 code structure layer 이고, `source_pointer`는 disk에 이미 존재하는 외부 자료를 다루는 external evidence layer 이다. `runtime_pointer`는 agent 실행 중 새로 생성되는 tool output을 다루는 runtime evidence layer이며, `self_extend`는 retrieval이 아니라 긴 prompt를 그대로 넣어보는 model execution fallback layer 이다. 실제 코드 어시스턴스에서는 `symbol_index`, `source_pointer`, `runtime_pointer`가 기본 경로이고, `self_extend`는 기본 경로가 실패하거나 비교 실험을 할 때만 제한적으로 사용한다.

네 항목은 모두 같은 시점에 한꺼번에 동작하지 않는다. `symbol_index`는 repository 구조를 미리 또는 변경 시점에 준비하는 장기 cache이고, `source_pointer`는 허용된 disk 자료를 필요할 때 등록하는 evidence catalog이며, `runtime_pointer`는 tool 실행 이후 생긴 큰 결과를 session 안에서 보관하는 임시 pointer이다. 반면 `self_extend`는 어떤 데이터를 저장하거나 index하는 구성요소가 아니라, 특정 LLM 호출에서만 선택되는 fallback 실행 옵션이다. 실제 운영 시점은 다음과 같이 구분한다.

항목	생성 및 갱신 타이밍	실제 운영 원칙
<code>symbol_index</code>	workspace를 열 때 또는 첫 질문이 들어왔을 때 초기 생성한다. 이후 파일 저장, 새 파일 생성, 파일 삭제, rename, branch checkout, agent가 만든 코드 파일 기록 직후 변경된 파일만 다시 parse 한다.	매 prompt마다 전체 repository를 다시 index하지 않는다. 파일별 hash, mtime, parser version을 cache key로 두고 dirty file만 incremental update한다. Parser 오류나 대량 변경이 감지되면 전체 재생성을 fallback으로 사용한다.
<code>source_pointer</code>	사용자가 경로를 언급하거나, project 설정에 등록된 source root를 scan하거나, 최근 test/build/report 산출물이 disk에 저장될 때 manifest에 등록한다.	disk의 모든 파일에 대해 미리 pointer를 만들지 않는다. 보안 allowlist 안의 log, markdown, JSON, CSV, report처럼 코드 어시스턴스에 필요한 자료만 catalog화한다. 파일 mtime/hash가 바뀌면 해당 manifest entry만 갱신한다.

<code>runtime_pointer</code>	tool이 실행되어 output을 반환하는 순간 생성 여부를 결정한다. test log, build log, API response, static analysis report처럼 크거나 재사용 가능하거나 구조화된 output이면 store에 저장하고 pointer를 발급한다.	runtime에 생기는 모든 데이터에 pointer를 만들지는 않는다. 작은 문자열은 그대로 LLM에 전달하고, threshold를 넘거나 이후 단계에서 다시 참조될 가능성이 높은 값만 <code>RuntimeStore</code> 에 저장한다. pointer는 session scope, TTL, LRU cleanup을 가진다.
<code>self_extend</code>	기본 pointer 경로가 context를 충분히 줄이지 못했거나, broad summarization처럼 긴 입력 비교가 필요한 단일 LLM 호출에서만 쓴다.	persistent index나 catalog가 아니다. per-call 옵션으로만 사용하며, 실행 뒤에는 파일명, symbol, line, test failure 같은 구체적 근거를 validation해야 한다. 검증에 실패하면 code patch로 이어가지 않고 pointer 기반 축소 경로로 되돌아간다.

3.2.1 symbol_index: repository 내부 코드 접근

`symbol_index`는 코드베이스 전체를 prompt에 넣지 않기 위한 첫 번째 단계이다. 이 단계는 `symbol_test/symbol_bench/indexer.py`에 구현되어 있으며, Python AST parser를 사용한다. 실험에서는 benchmark 실행 전에 corpus를 만들고 그 시점의 repository snapshot에 대해 index를 한 번 생성했다. 그러나 실제 코드 어시스턴스에서는 workspace를 열 때 background job으로 만들거나, 첫 질문이 들어왔을 때 lazy build한 뒤 cache해야 한다. 이후 사용자가 파일을 저장하거나 agent가 새 코드를 생성하면 해당 파일만 다시 parse해 index를 갱신한다. 이렇게 해야 LLM이 오래된 symbol 위치를 보지 않으면서도, 모든 질문마다 전체 repository를 다시 읽는 비용을 피할 수 있다.

중요한 점은 index freshness를 LLM의 판단에 맡기지 않는 것이다. LLM은 “이 파일이 방금 수정되었는지”를 안정적으로 알 수 없으므로, system layer가 file watcher, mtime/hash, branch 변경 이벤트를 보고 index version을 관리해야 한다. 예를 들어 agent가 `services/auth_service.py`를 생성하거나 수정한 직후에는 patch 적용이 끝난 시점에 해당 파일을 즉시 reparse하고, selector가 보는 `memory://symbol/...` URI도 새 line range를 가리키도록 바꾼다. 반대로 사용자가 아무 파일도 바꾸지 않았다면 같은 index cache를 재사용한다. 동작 순서는 다음과 같다.

1. **입력:** repository의 Python 파일들이다. 실험에서는 `symbol_test/work/medium/corpus/` 아래의 27개 파일을 사용했다.
2. **AST parsing:** `build_index`가 각 파일을 `ast.parse`로 분석한다. `_SymbolVisitor`는 `ClassDef`, `FunctionDef`, `AsyncFunctionDef`를 방문한다.
3. **Index 생성:** 각 symbol에 대해 file path, kind, name, qualname, start line, end line, URI를 기록한다.
4. **Selector 입력 축소:** `compact_outline` 또는 `_candidate_symbol_outline`이 전체 source code 대신 symbol metadata만 남긴다.
5. **Symbol 선택:** selector가 질문에 맞는 `memory://symbol/...` URI를 고른다. Oracle 실험에서는 정답 URI를 이미 알고 있다고 가정했다.
6. **Symbol resolve:** `resolve_symbol`은 선택된 URI의 line range를 읽는다. Hard task에서는 `expand_symbol_context`가 primary symbol뿐 아니라 같은 파일의 상수, helper/callee, 질문에 언급된 관련 symbol까지 bundle로 확장한다.
7. **LLM 입력:** 최종적으로 LLM에는 전체 repository가 아니라 선택된 symbol bundle만 들어간다.

AST parser가 생성한 실제 `symbol_index.json` 항목 예시는 다음과 같다.

```
{
  "path": "services/auth_service.py",
  "kind": "function",
  "name": "validate_token",
  "qualname": "validate_token",
  "start_line": 19,
  "end_line": 26,
  "uri": "memory://symbol/services/auth_service.py::validate_token"
}
```

이 index만으로는 함수의 위치를 알 수 있지만, hard task처럼 함수 밖 상수까지 필요한 질문에는 부족하다. 따라서 `expand_symbol_context`가 AST를 다시 분석해 primary symbol 내부에서 참조한 이름을 찾고, 같은 파일의 module-level assignment를 bundle에 포함한다. 실제 `validate_token` 예시에서 생성된 bundle metadata는 다음과 같다.

```
{
  "primary_uri": "memory://symbol/services/auth_service.py::validate_token",
  "bundle_uris": [
    "memory://symbol/services/auth_service.py::validate_token"
  ],
  "bundle_assignments": [
    {
      "path": "services/auth_service.py",
      "names": ["TOKEN_EXPIRED_MESSAGE"],
      "start_line": 3,
      "end_line": 3
    },
    {
      "path": "services/auth_service.py",
      "names": ["TOKEN_MISSING_USER_MESSAGE"],
      "start_line": 4,
      "end_line": 4
    }
  ],
  "bundle_fragment_count": 3
}
```

LLM에 실제로 전달되는 bundle context는 다음처럼 provenance와 source를 함께 가진다.

```
### module-level name referenced by selected symbol:
### services/auth_service.py#L3-L3 (TOKEN_EXPIRED_MESSAGE)
0003: TOKEN_EXPIRED_MESSAGE = "TOKEN_EXPIRED"

### primary symbol:
### memory://symbol/services/auth_service.py::validate_token
0019: def validate_token(token: dict, now: int) -> str:
0020: if not token.get("active", False):
0021: return "inactive"
0022: if token.get("expires_at", 0) <= now:
0023: raise ValueError(TOKEN_EXPIRED_MESSAGE)
```

이 구조에서 각 레벨의 출력은 명확하다. AST parser는 `symbol_index.json`을 출력하고, selector는 `memory://symbol/...` URI를 출력하며, resolver는 line-numbered source fragment를 출력한다. Bundle expander는 여러 fragment를 provenance와 함께 묶어 LLM의 최종 evidence context로 전달한다.

3.2.2 source_pointer: disk 외부 자료 접근

`source_pointer`는 repository 밖 또는 repository 안에 저장된 큰 외부 자료를 raw context로 넣지 않기 위한 계층이다. 실험 코드는 `source_pointer_test/source_pointer_bench/source_tools.py`

에 있으며, 대상 데이터는 log, markdown document, JSON이다. 실험에서는 `make_corpus.py`가 synthetic source file을 만든 뒤 모든 실험 대상 파일에 대해 `manifest.json`을 미리 생성했다. 하지만 실제 코드 어시스턴스가 disk의 모든 파일을 미리 scan해 pointer를 만드는 방식은 적절하지 않다. 보안상 사용자 home directory나 임의 경로 전체를 catalog화하면 안 되고, 성능상으로도 대부분의 파일은 현재 질문과 무관하기 때문이다.

실제 운영에서는 project 설정이나 사용자 명시 경로로 허용된 source root만 다룬다. 예를 들어 `logs/`, `reports/`, `docs/`, `snapshots/`처럼 코드 어시스턴스가 참조할 수 있는 directory를 allowlist로 등록하고, 그 안에서도 log, markdown, JSON, CSV, test report처럼 extractor가 있는 file kind만 pointer 후보로 둔다. Pointer는 세 시점에 생성된다. 첫째, 사용자가 질문에서 특정 파일 경로를 언급했을 때이다. 둘째, project 설정에 등록된 source root를 background catalog job이 발견했을 때이다. 셋째, test나 build가 disk에 report를 남겨 다음 단계에서 참조할 가능성이 생겼을 때이다. 이미 등록된 파일은 mtime/hash가 바뀌었을 때만 manifest entry를 갱신한다. 동작 순서는 다음과 같다.

1. **입력:** disk에 존재하는 source file이다. 실험에서는 checkout log, operations runbook, tenant snapshot JSON을 생성했다.
2. **Manifest 생성:** `make_corpus.py`가 각 file의 kind, relative path, byte size, pointer, 사용 가능한 tool, hint를 `manifest.json`에 기록한다.
3. **Catalog 구성:** `compact_catalog`는 LLM selector에 전달할 작은 catalog를 만든다. 이때 file 전체 내용은 전달하지 않는다.
4. **Tool 선택:** `pointer_select` mode에서는 LLM이 catalog와 tool spec을 보고 `grep_log`, `extract_doc_section`, `extract_json_field` 중 하나와 인자를 JSON으로 출력한다.
5. **Pointer resolve:** `resolve_pointer`가 `source://...` URI를 manifest에서 찾고, `sources/` root 밖으로 path가 벗어나지 않는지 검사한다.
6. **Extraction:** 선택된 tool이 필요한 line, section, JSON path만 추출한다.
7. **LLM 입력:** 최종 답변 LLM 호출에는 전체 source file이 아니라 extractor output만 들어간다. 실제 `manifest.json`의 일부는 다음과 같다.

```
{
  "kind": "log",
  "rel_path": "logs/checkout_gateway.log",
  "pointer": "source://log/logs/checkout_gateway.log",
  "bytes": 79072,
  "tools": ["grep_log"],
  "hints": ["request_id", "payment_gateway", "retry_after_ms", "reason"]
}
{
  "kind": "json",
  "rel_path": "json/tenant_snapshot.json",
  "pointer": "source://json/json/tenant_snapshot.json",
  "bytes": 115290,
  "tools": ["extract_json_field"],
  "hints": [
    "tenants.northwind-prod.security",
    "source_pointer_enabled",
    "max_context_policy"
  ]
}
```

`pointer_select` 실험에서 JSON tenant policy 질문에 대해 selector가 출력한 tool call은 다음과 같다.

```
{
  "tool": "extract_json_field",
  "args": {
    "pointer": "source://json/json/tenant_snapshot.json",
    "path": "tenants.northwind-prod.security.source_pointer_enabled,max_context_policy"
  }
}
```

이 tool call은 run_tool을 통해 실행되고, extractor는 큰 JSON 전체가 아니라 필요한 field만 반환한다.

```
{
  "max_context_policy": "pointer-first",
  "source_pointer_enabled": true
}
```

그 다음 LLM은 이 작은 결과만 보고 다음 답을 생성했다.

```
enabled=true, max_context_policy=pointer-first
```

즉 source pointer 계층에서 중간 산출물은 manifest.json, compact catalog, selector tool call JSON, extractor output이다. 이 네 단계가 분리되어 있기 때문에 LLM은 115,290 byte JSON을 직접 읽지 않고도 필요한 값을 얻을 수 있다.

3.2.3 runtime_pointer: 실행 중 생성된 tool output 접근

runtime_pointer는 source pointer와 유사하지만, 대상이 disk file이 아니라 agent 실행 중 생성된 runtime object라는 점이 다르다. 예를 들어 test 실행 결과, build log, API response, static analysis report는 사용자가 처음부터 제공한 file이 아니라 tool 실행 이후에 생긴다. 실험 코드는 runtime_pointer_test/runtime_pointer_bench/runtime_tools.py에 있다. 실험에서는 producer를 실행하면 큰 log/doc/JSON output을 항상 RuntimeStore에 저장하고 pointer catalog를 만들었다. 실제 agent에서는 더 선택적으로 동작해야 한다.

Runtime pointer는 미리 만들 수 없다. 어떤 runtime data가 생길지는 사용자가 어떤 명령을 실행하는지, 테스트가 성공하는지, tool이 어떤 응답을 돌려주는지에 따라 달라지기 때문이다. 따라서 생성 시점은 tool output이 반환되는 순간이다. 이때 system layer가 output 크기, type, 재사용 가능성, 민감도에 따라 세 가지 중 하나를 선택한다. 출력이 짧고 안전하면 그대로 LLM message에 넣는다. 출력이 길거나 JSON/list/table처럼 구조화되어 있거나 이후 단계에서 다시 참조될 가능성이 있으면 RuntimeStore에 저장하고 runtime://... pointer와 compact summary만 반환한다. 출력에 secret이나 credential이 섞일 가능성이 있으면 raw text를 그대로 prompt에 넣지 않고, hint를 축약하거나 mask한 뒤 extractor를 통해 필요한 부분만 노출한다.

Runtime pointer는 session-local resource이다. 같은 대화 안에서는 이후 selector와 extractor가 runtime://log/run_tests/0001 같은 pointer를 다시 사용할 수 있지만, 사용자가 새 session을 시작하거나 TTL이 지나면 정리된다. 너무 많은 runtime object가 쌓이면 LRU 정책으로 오래된 값을 제거하고, 사용자가 명시적으로 보존한 report만 disk source pointer로 승격할 수 있다. 동작 순서는 다음과 같다.

1. **Runtime producer 실행:** collect_checkout_trace, compile_operations_runbook, fetch_tenant_snapshot 같은 producer가 큰 문자열 또는 dict를 만든다.
2. **RuntimeStore 저장:** output이 크면 RuntimeStore.put이 값을 process-local store에 저장하고 runtime://kind/producer/id pointer를 발급한다.
3. **Catalog 출력:** RuntimeStore.catalog는 pointer, producer, kind, byte size, tools, hints를 가진 작은 catalog를 만든다.
4. **Tool 선택:** runtime_pointer_select mode에서는 LLM이 catalog와 tool spec을 보고 grep_runtime_log, extract_runtime_doc_section, extract_runtime_json_path 중 하나를 선택한다.

5. **Pointer resolve:** runtime tool wrapper가 pointer를 `RuntimeStore.resolve`로 원본 object에 대응시킨다.
6. **Extraction:** 원본 object 전체가 아니라 필요한 line, section, JSON path만 추출한다.
7. **LLM 입력:** LLM에는 extractor output과 질문만 전달된다.

실제 runtime JSON producer를 실행하면 catalog는 다음과 같은 형태가 된다.

```
{
  "pointer": "runtime://json/fetch_tenant_snapshot/0001",
  "producer": "fetch_tenant_snapshot",
  "kind": "json",
  "bytes": 134023,
  "tools": ["extract_runtime_json_path"],
  "hints": [
    "tenants.northwind-prod.security",
    "runtime_pointer_enabled",
    "max_context_policy"
  ]
}
```

`runtime_pointer_select` 실험에서 selector가 고른 tool call은 다음과 같다.

```
{
  "tool": "extract_runtime_json_path",
  "args": {
    "pointer": "runtime://json/fetch_tenant_snapshot/0001",
    "path": "tenants.northwind-prod.security.runtime_pointer_enabled,max_context_policy"
  }
}
```

이 tool call을 실행하면 134,023 characters의 runtime JSON에서 다음 78 characters 수준의 결과만 추출된다.

```
{
  "max_context_policy": "pointer-first",
  "runtime_pointer_enabled": true
}
```

실험 결과 row에서는 같은 과정이 다음처럼 기록되었다.

```
{
  "memory_pointer": "runtime://json/fetch_tenant_snapshot/0001",
  "raw_runtime_chars": 134023,
  "tool_output_chars": 78,
  "prompt_tokens": 598,
  "wall_seconds": 6.7198,
  "answer_text": "enabled=true, max_context_policy=pointer-first"
}
```

Runtime pointer는 실제 코드 어시스턴스에서 특히 중요하다. 테스트를 실행한 뒤 수만 줄의 실패 로그가 생겼을 때, 그 로그 전체를 message history에 넣으면 이후 모든 LLM 호출이 비싸지고 불안정해진다. 반대로 runtime pointer로 저장하면 실패 block, stack frame, assertion 주변부만 추출해 다음 단계의 symbol selector로 넘길 수 있다.

3.2.4 self_extend: 긴 context 직접 수용 fallback

`self_extend`는 앞의 세 pointer 계층과 성격이 다르다. Pointer 계층은 context를 줄이는 구조이고, Self-Extend는 긴 context를 모델이 직접 받도록 attention 설정을 바꾸는 방법이다. 실험 코드는 `self_extend_test/run_llama2_selfextend_passkey.py`, `self_extend_test/run_code_assistance_bench`에 있다.

`self_extend`는 `symbol_index`, `source_pointer`, `runtime_pointer`처럼 사전에 만들어두거나 갱신하는 구조가 아니다. 이것은 특정 LLM 호출에서 context 수용 방식을 바꾸는 실행 옵션이다. 따라서 실제 코드 어시스턴스에서는 항상 켜두지 않는다. 기본 경로는 먼저 `symbol/source/runtime pointer`로 evidence를 줄이는 것이고, 그 결과도 여전히 너무 크거나 사용자가 repository 전체 요약처럼 넓은 범위의 질문을 했을 때만 fallback으로 사용한다. 또한 Self-Extend 실행 결과가 곧바로 code patch로 이어지면 안 된다. 출력이 어떤 파일, 어떤 symbol, 어떤 line, 어떤 test failure에 근거하는지 validation 단계에서 확인하고, 구체적인 근거를 제시하지 못하면 다시 pointer 기반 검색으로 돌아가야 한다.

동작 순서는 다음과 같다.

1. **Prompt 구성:** passkey 실험은 긴 junk text와 passkey 질문을 만들고, code assistance benchmark는 synthetic repository 전체를 하나의 prompt로 만든다.
2. **Context 설정:** off 조건은 기본 context를 사용한다. G=2, G=4 조건은 llama-completion 또는 llama-passkey에 `--grp-attn-n 2`, `--grp-attn-n 4`를 전달한다.
3. **Model 실행:** llama.cpp가 실제 `n_ctx`, prompt token, 출력, 성능 로그를 남긴다.
4. **결과 판정:** passkey가 출력에 포함되는지, 코드 benchmark의 expected substring이 답에 포함되는지 평가한다.

Passkey 실험 결과 일부는 다음과 같다.

```
case: q4_k_m_off_junk180_pos90_seed1
group: off
prompt_tokens: 4387
n_ctx: 4320
failure: context_slot_overflow
correct: false

case: q4_k_m_g2_junk320_pos160_seed1
group: G=2
prompt_tokens: 7747
n_ctx: 8416
answer: "<repeated U+0409 characters>"
correct: false

case: q4_k_m_g4_junk320_pos160_seed1
group: G=4
prompt_tokens: 7747
n_ctx: 16608
answer: "<unk><unk><unk>..."
correct: false
```

Code assistance benchmark의 결과 row도 같은 패턴을 보인다.

```
{
  "case": "q4_k_m_off_function_definition",
  "ctx_size": 4096,
  "failure": "prompt_too_long",
  "correct": false,
  "estimated_prompt_tokens": 3080
}
{
  "case": "q4_k_m_g2_function_definition",
  "ctx_size": 8192,
  "prompt_tokens": 4854,
  "answer": "<repeated U+0409 character>",
  "correct": false,
  "total_ms": 17758.39
}
```

따라서 Self-Extend 계층의 중간 산출물은 pointer나 extractor output이 아니라, 긴 prompt 실행 로그이다. 구체적으로는 `n_ctx`, `prompt_tokens`, `answer`, `failure`, `correct`가 산출물이다. 이 실험은 Self-Extend가 context overflow를 일부 회피할 수 있더라도, 코드 어시스턴스에서 필요한 정확한 retrieval과 reasoning을 보장하지 못한다는 것을 보여준다. 그래서 제안 시스템에서는 Self-Extend를 기본 경로로 두지 않고, pointer 기반으로 context를 줄인 뒤에도 실패하는 경우 비교 또는 fallback 목적으로만 사용한다.

3.3 개선점: 기존 시스템의 문제 해결

기존 full-context 방식의 문제는 context overflow, 긴 prompt eval, distractor로 인한 오답이다. 제안 시스템은 이를 다음 방식으로 해결한다.

1. **전체 repository 대신 symbol 접근:** Python AST parser로 파일별 함수와 class를 index하고, `memory://symbol/file.py::qualname` URI로 필요한 symbol만 읽는다.
2. **단일 symbol 한계 보완:** `expand_symbol_context`는 선택된 symbol뿐 아니라 같은 파일의 module-level constant, helper/callee, 질문에 명시된 관련 symbol을 bundle로 확장한다.
3. **전체 외부 파일 대신 source pointer:** log, markdown, JSON 파일을 `source://` pointer catalog로 관리하고, `grep_log`, `extract_doc_section`, `extract_json_field`로 필요한 부분만 추출한다.
4. **전체 tool output 대신 runtime pointer:** 큰 tool output은 RuntimeStore에 저장하고, LLM에는 `runtime://` pointer와 compact catalog만 제공한다.
5. **Naive selector 비용 축소:** 전체 symbol index를 LLM에 넣지 않고 질문 기반 후보 필터링 후 작은 후보 목록만 selector에 전달한다.
6. **Retrieval과 reasoning 분리:** pointer resolver와 extractor는 deterministic하게 데이터를 줄이고, LLM은 작은 evidence bundle을 보고 reasoning한다. 문자열 계산이나 산술 계산은 향후 evaluator로 보조한다.

3.4 차별점: 추가된 기술과 기능

기존 시스템의 차별점은 code pointer, source pointer, runtime pointer를 하나의 코드 어시스턴스 workflow로 결합한다는 점이다. 기존 agent context overflow 논문은 runtime tool output을 memory pointer로 처리한다. 본 연구는 그 아이디어를 코드 어시스턴스에 맞게 다음 세 영역으로 확장한다.

- **Code structure layer:** `symbol_test/symbol_bench/indexer.py`는 AST 기반으로 `build_index`, `compact_outline`, `resolve_symbol`, `read_span`, `expand_symbol_context`를 제공한다.
- **External evidence layer:** `source_pointer_test/source_pointer_bench/source_tools.py`는 `source://` URI 검증, manifest lookup, source file resolver, format-specific extractor를 제공한다.
- **Runtime evidence layer:** `runtime_pointer_test/runtime_pointer_bench/runtime_tools.py`는 RuntimeStore, runtime pointer resolver, runtime log/doc/JSON extractor를 제공한다.

또한 각 fragment에는 path, line range, URI, 포함 이유를 남긴다. 이 provenance는 LLM이 어떤 근거로 답변했는지 추적하고, 코드 수정 단계에서 어떤 파일을 수정해야 하는지 결정하는 데 사용된다.

3.5 목적과 구현 방식

제안 시스템의 전체 workflow는 다음과 같다.

1. 사용자가 자연어 요청을 입력한다.
2. 요청에 stack trace, test name, error message, file path, function name이 포함되어 있는지 분석한다.
3. repository symbol index와 rg 검색 결과를 사용해 후보 파일과 symbol을 줄인다.
4. 후보 symbol outline을 LLM selector 또는 deterministic matcher에 전달한다.
5. 선택된 symbol에 대해 `expand_symbol_context`를 실행해 상수, helper, callee, 관련 symbol을 bundle로 구성한다.
6. 외부 log, document, JSON이 필요하면 `source://` pointer와 extraction tool을 사용한다.
7. test 실행이나 build 실행으로 큰 output이 생성되면 `runtime://` pointer로 저장하고 필요한 부분만 추출한다.
8. LLM은 작은 evidence bundle만 보고 원인 분석, 답변, patch plan을 생성한다.
9. 코드 수정 후 테스트를 실행하고, 실패 output은 다시 runtime pointer로 관리한다.

이 구조의 핵심은 LLM context를 “전체 데이터 저장소”로 쓰지 않는 것이다. Context는 현재 reasoning에 필요한 작은 증거 묶음으로 유지된다.

4 Evaluation

4.1 실험 환경

실험은 로컬 환경에서 진행되었다. symbol, source pointer, runtime pointer 실험은 Apple M3 MacBook Pro, llama.cpp Metal backend, Qwen2.5 Coder 7B Instruct GGUF Q5_K_M 모델을 사용했다. Self-Extend 실험은 Llama-2-7B-chat GGUF Q4_K_M과 Q5_K_M을 사용했다. 대부분의 generation 설정은 temperature 0, seed 1로 고정했다.

본 절의 모든 실험 코드는 실험 결과 markdown 파일과 동일한 이름의 폴더에 위치한다. 예를 들어 Documents/Experiments_result/source_pointer_test.md의 코드는 source_pointer_test/에 있다.

4.2 Symbol Index와 Symbol Bundle 실험

4.2.1 비교 방식

Symbol 실험은 repository 내부 코드를 어떤 단위로 LLM에게 전달하는지가 성능에 미치는 영향을 측정한다. 비교 방식은 다음과 같다.

Mode	설명
full_repo	모든 Python 파일 내용을 prompt에 직접 삽입한다. 구현은 단순하지만 repository가 커지면 context overflow와 prompt eval 지연이 발생한다.
full_file	정답이 들어 있는 파일 하나만 통째로 prompt에 넣는다. 전체 repo보다 효율적이지만 agent가 이미 올바른 파일을 알고 있다는 가정이 필요하다.
line_span	정답 함수의 line range만 prompt에 넣는다. 매우 작고 빠르지만 line 번호를 정확히 알고 있어야 한다.

symbol_oracle	정답 symbol URI를 이미 알고 있다고 가정하고 resolve_symbol로 해당 함수 또는 class 본문만 읽는다. Symbol 접근의 이상적 성능을 측정하기 위한 방식이다.
symbol_select	LLM에게 전체 symbol index를 보여주고 필요한 symbol URI를 고르게 한다. 실제 agent workflow에 가깝지만 전체 index를 넣기 때문에 selector prompt가 커진다.
symbol_bundle_oracle	정답 symbol URI를 이미 알고 있다고 가정하되, 단일 symbol이 아니라 expand_symbol_context로 상수와 helper를 함께 읽는다.
symbol_filtered_select_bundle	질문 기반 후보 필터링 후 작은 후보 목록에서 symbol을 선택하고, 선택된 symbol을 bundle로 확장한다. 제안 시스템에 가장 가까운 방식이다.

4.2.2 쉬운 질문 결과

Small corpus의 1개 쉬운 질문에서는 모든 방식이 정답을 맞혔다. 그러나 token과 시간은 크게 달랐다.

Mode	Accuracy	Wall time	Prompt tokens	Prompt eval
full_repo	1.00	14.200s	4098	13056.17ms
full_file	1.00	2.089s	317	985.53ms
line_span	1.00	2.110s	217	694.36ms
symbol_oracle	1.00	2.109s	214	683.56ms
symbol_select	1.00	11.304s	2709	8390.18ms
symbol_bundle_oracle	1.00	5.052s	302	973.31ms
symbol_filtered_select_bundle	1.00	4.766s	728	2268.92ms

symbol_oracle은 full_repo 대비 prompt token을 약 94.8%, wall time을 약 85.1% 줄였다. symbol_select는 정답 symbol을 선택했지만 selector 호출 때문에 작은 corpus에서는 oracle보다 느렸다. 추가 측정한 bundle 계열도 정답은 맞혔다. 다만 쉬운 질문은 단일 함수 또는 단일 상수 수준에서 답이 나오므로 bundle 확장의 장점이 크게 드러나지 않는다. symbol_bundle_oracle은 상수와 주변 fragment를 함께 넣어 prompt token이 302로 증가했고, symbol_filtered_select_bundle은 selector 호출까지 포함해 728 token을 사용했다. 그럼에도 두 방식 모두 full_repo와 naive symbol_select보다 작은 prompt로 동작했다.

4.2.3 Medium corpus 결과

Medium corpus는 27개 파일, 638개 symbol, 3개 task로 구성되었다. full_repo는 65k context에서 prompt is too long으로 실패했고, 131k context에서 첫 케이스만 재측정했다.

Mode	Cases	Accuracy	Mean wall	Mean tokens	Selection acc.
full_repo	1	1.000	612.869s	67188	—
full_file	3	1.000	2.286s	256	—
line_span	3	1.000	2.120s	199	—
symbol_oracle	3	1.000	2.131s	199	—
symbol_select	3	1.000	181.191s	33871	1.000
symbol_bundle_oracle	3	1.000	2.600s	305	—
symbol_filtered_select_bundle	3	1.000	6.262s	824	1.000

결과적으로 symbol_oracle은 full_repo 대비 prompt token과 wall time을 모두 약 99.7% 줄였다. 반면 naive symbol_select는 정답률과 selection accuracy가 1.000이지만, 전체 638개

symbol index를 매번 prompt에 넣어 평균 181.191초가 걸렸다. 이는 selector 입력을 줄이지 않으면 symbol 접근 자체의 장점이 사라진다는 것을 보여준다.

추가측정한 bundle 계열 결과는 제안 시스템의 실제 동작 비용을 더 잘 보여준다. `symbol_bundle_oracle`은 단일 symbol보다 많은 context를 넣기 때문에 평균 prompt token이 305로 증가했지만, 평균 wall time은 2.600초로 여전히 매우 작았다. `symbol_filtered_select_bundle`은 selector와 bundle answer 호출을 모두 포함해 평균 824 token, 6.262초가 걸렸다. 이는 `symbol_oracle`보다는 느리지만, naive `symbol_select`의 33,871 token과 181.191초에 비해 prompt token은 약 97.6%, wall time은 약 96.5% 감소한 값이다. 따라서 후보 symbol을 먼저 줄인 뒤 bundle을 확장하는 방식은 medium 규모에서도 정확도를 유지하면서 selector 비용을 현실적인 수준으로 낮춘다.

4.2.4 Hard task 결과

Hard task는 단일 함수 본문만으로는 답을 알 수 없도록 구성되었다. 예를 들어 `validate_token`의 예외 literal은 함수 내부가 아니라 module-level constant에 있고, `build_cache_key`의 최종 결과는 helper 함수 `sanitize_report_name`까지 읽어야 계산할 수 있다.

Mode	Cases	Accuracy	Mean wall	Mean tokens	Selection acc.
<code>full_repo</code>	3	0.000	1.087s	52313	—
<code>full_file</code>	3	0.333	2.604s	281	—
<code>line_span</code>	3	0.000	2.461s	224	—
<code>symbol_oracle</code>	3	0.000	2.451s	225	—
<code>symbol_select</code>	3	0.000	172.170s	33922	1.000

`symbol_select`는 3개 질문 모두 대표 symbol URI를 올바르게 선택했지만 최종 답은 모두 실패했다. 즉 이 결과에서 병목은 “대표 symbol을 찾는 것”이 아니라 “대표 symbol 하나만 읽었을 때 필요한 주변 정보가 누락되는 것”이었다. 실제 코드 이해 질문은 상수, helper 함수, class method, 관련 함수 호출을 함께 봐야 하는 경우가 많으므로 hard task 결과는 단일 `read_symbol` 구조의 한계를 보여준다.

이를 확인하기 위해 hard task 결과에는 `symbol_bundle_oracle`과 `symbol_filtered_select_bundle`을 추가했다. 이 두 mode는 대표 symbol을 찾은 뒤 `expand_symbol_context`를 실행하여 module-level constant, same-file helper/callee, 질문에 직접 언급된 관련 symbol을 bundle로 묶는다.

Mode	Cases	Accuracy	Mean wall	Mean tokens	Selection acc.
<code>symbol_bundle_oracle</code>	3	0.333	2.298s	369	—
<code>symbol_filtered_select_bundle</code>	3	0.333	4.588s	587	1.000

Hard task에서 `symbol_filtered_select_bundle`은 기존 `symbol_select`와 비교해 prompt token을 33,922에서 587로 줄였다. 이는 약 98.3% 감소이다. Wall time도 172.170초에서 4.588초로 줄어 약 97.3% 감소했다. Selection accuracy는 1.000으로 유지되었다.

다만 정확도는 1/3에 그쳤다. 실패 원인은 retrieval 부족이 아니라 reasoning 오류였다. `build_cache_key` task에서는 helper가 bundle에 포함되었지만 모델이 slash를 underscore로 잘못 바꾸었고, `billing` task에서는 10000×0.0125 계산을 125가 아니라 150으로 답했다. 따라서 제안 시스템은 다음 단계에서 deterministic evaluator를 추가해야 한다.

4.3 Source Pointer 실험

4.3.1 비교 방식

Source pointer 실험은 disk에 존재하는 큰 외부 데이터를 LLM context에 직접 넣는 방식과 pointer 기반 extraction 방식을 비교한다. 외부 데이터는 log, markdown document, JSON으로 구성되었다.

Mode	설명
full_source	외부 파일 전체를 prompt에 직접 삽입한다. Log와 JSON이 크면 context overflow가 발생한다.
pointer_oracle	필요한 tool call을 이미 알고 있다고 가정하고, extractor가 필요한 일부만 추출해 prompt에 넣는다.
pointer_select	LLM이 compact source catalog와 tool spec을 보고 tool call JSON을 선택한 뒤, extractor 결과로 답변한다.

사용한 extraction tool은 `grep_log`, `extract_doc_section`, `extract_json_field`이다. 이는 데이터 형식별로 자연스러운 접근 방식을 제공한다.

4.3.2 결과 분석

Scale	Mode	Accuracy	Mean wall	Mean tokens	Selection acc.
Small	full_source	0.333	54.401s	18369	—
Small	pointer_oracle	1.000	2.472s	286	—
Small	pointer_select	1.000	6.430s	778	1.000
Medium	full_source	0.333	22.117s	120208	—
Medium	pointer_oracle	1.000	2.443s	286	—
Medium	pointer_select	1.000	6.423s	780	1.000

Small에서 full_source는 log task가 35,923 token으로 32,764 max를 초과해 실패했다. JSON task는 context 안에는 들어갔지만 distractor 값인 pointer-preferred를 답해 실패했다. Medium에서는 log와 JSON이 각각 275,609 token, 216,172 token으로 context를 초과했다.

Medium 기준으로 pointer_oracle은 full_source 대비 prompt token을 120,208에서 286으로 줄였다. 이는 약 99.8% 감소이다. pointer_select도 780 token만 사용해 약 99.4% 감소를 보였다. Wall time은 pointer_oracle이 22.117초에서 2.443초로 줄어 약 89.0% 감소했고, pointer_select는 6.423초로 약 71.0% 감소했다. 특히 medium document task의 성공 workflow만 보면 full_source는 16,770 token과 63.68초가 필요했지만, pointer_oracle은 217 token과 2.12초로 끝났다.

이 결과는 source pointer가 단순한 token 절약 기술이 아니라 외부 데이터 크기와 LLM context 크기를 분리하는 구조임을 보여준다.

4.4 Runtime Pointer 실험

4.4.1 비교 방식

Runtime pointer 실험은 실행 중 생성되는 큰 tool output을 다룬다. Source pointer가 disk에 이미 있는 외부 파일을 가리킨다면, runtime pointer는 현재 agent session에서 tool이 생성한 중간 결과를 가리킨다.

Mode	설명
------	----

<code>full_runtime</code>	runtime tool output 전체를 prompt에 직접 삽입한다.
<code>runtime_pointer_oracle</code>	필요한 mirrored extraction tool call을 이미 알고 있다고 가정하고, 추출 결과만 prompt에 넣는다.
<code>runtime_pointer_select</code>	LLM이 runtime memory catalog와 tool spec을 보고 tool call을 선택한 뒤, 추출 결과로 답변한다.

Runtime object는 RuntimeStore에 저장되며, pointer는 `runtime://log/collect_checkout_trace/0001` 같은 형식을 가진다. 사용한 tool은 `grep_runtime_log`, `extract_runtime_doc_section`, `extract_runtime_json_path`이다.

4.4.2 결과 분석

Scale	Mode	Accuracy	Mean wall	Mean tokens	Selection acc.	Tool chars
Small	<code>full_runtime</code>	0.333	4.961s	18785	–	0
Small	<code>runtime_pointer_oracle</code>	1.000	2.768s	311	–	365
Small	<code>runtime_pointer_select</code>	1.000	7.050s	756	1.000	430
Medium	<code>full_runtime</code>	0.333	26.270s	131136	–	0
Medium	<code>runtime_pointer_oracle</code>	1.000	2.610s	311	–	365
Medium	<code>runtime_pointer_select</code>	1.000	6.891s	757	1.000	430

Small에서도 `full_runtime`은 log와 JSON task에서 context overflow로 실패했다. Medium에서는 log가 275,619 token, JSON이 238,593 token으로 32,764 max를 초과했다. 반면 pointer 방식은 small과 medium 모두 3/3 성공했다.

Medium 기준으로 `runtime_pointer_oracle`은 prompt token을 131,136에서 311로 줄여 약 99.8% 감소를 보였고, wall time은 26.270초에서 2.610초로 줄어 약 90.1% 감소를 보였다. `runtime_pointer_select`는 757 token과 6.891초를 사용해 prompt token 약 99.4%, wall time 약 73.8% 감소를 보였다. 또한 데이터 크기가 small에서 medium으로 커져도 pointer mode의 prompt token은 거의 변하지 않았다. `runtime_pointer_oracle`은 small과 medium 모두 311 token이고, `runtime_pointer_select`는 756에서 757 token으로만 증가했다.

이 결과는 runtime pointer가 test output, build log, API response, static analysis report 처럼 실행 중 생성되는 큰 데이터를 message history에 직접 넣지 않게 해준다는 것을 보여준다.

4.5 Self-Extend 실험

4.5.1 비교 방식

Self-Extend 실험은 context window를 늘리는 방식이 코드 어시스턴스의 기본 해결책이 될 수 있는지 확인하기 위해 수행했다. 비교 조건은 Self-Extend off, group attention G=2, G=4이다. 모델은 Llama-2-7B-chat Q4_K_M과 Q5_K_M을 사용했다.

실험은 두 종류로 구성되었다.

- **Passkey retrieval:** 긴 junk text 중간의 5자리 passkey를 회수하는 synthetic long-context task.
- **Code assistance benchmark:** 긴 repo context 안에서 함수 반환값, symbol reference, failing test 원인을 찾는 task.

4.5.2 결과 분석

Passkey 결과에서 Self-Extend는 `n_ctx`를 G=2에서 약 8416, G=4에서 약 16608까지 늘렸지만, 정답 회수에는 실패했다. 이는 context 수용량과 retrieval 품질이 분리된 문제임을 보여준다.

모델	조건	성공	해석
Q4_K_M	off	2/3	3907 token 은 성공, 4387 token 은 context overflow
Q4_K_M	G=2	0/6	입력은 수용했지만 passkey retrieval 실패
Q4_K_M	G=4	0/2	입력은 수용했지만 출력 붕괴
Q5_K_M	off	2/3	Q4와 동일한 패턴
Q5_K_M	G=2	0/6	Q4와 동일하게 retrieval 실패
Q5_K_M	G=4	0/2	빈 출력 또는 출력 붕괴

Code assistance benchmark에서는 2개 모델, 3개 조건, 3개 task를 합쳐 모든 조건이 실패했다. Off 조건은 약 4.8k token repo context를 4k context가 처리하지 못해 `prompt too long`으로 실패했고, G=2/G=4 조건은 입력을 수용했지만 함수 정의, symbol reference, failing test 원인을 모두 맞히지 못했다.

따라서 Self-Extend는 현재 환경에서 코드 어시스턴스 기본 retrieval 방식으로 부적합하다. 다만 긴 context를 넣어보는 실험적 fallback으로는 사용할 수 있다. 이 경우에도 수정 단계 전에 함수명, 파일명, 테스트명, 원인 expression을 별도로 회수하는 sanity check가 필요하다.

4.6 종합 평가

전체 실험의 결론은 명확하다. 로컬 LLM 기반 코드 어시스턴스에서는 full-context 방식보다 pointer와 retrieval 기반 방식이 훨씬 적합하다.

실험	핵심 결론
Symbol index	단일 symbol 접근은 빠르지만 hard task에서는 주변 상수와 helper가 필요하다. Filtered selector와 bundle 확장은 기존 <code>symbol_select</code> 대비 token 98.3%, wall time 97.3%를 줄였다.
Source pointer	disk 외부 데이터를 pointer와 extractor로 다루면 medium 기준 prompt token을 약 99.8% 줄이고 정확도를 1/3에서 3/3으로 높였다.
Runtime pointer	실행 중 생성되는 큰 tool output도 pointer로 관리하면 데이터 크기와 LLM context 크기를 분리할 수 있다. Medium 기준 oracle은 token 99.8%, wall time 90.1%를 줄였다.
Self-Extend	context 수용량 확장은 가능하지만 retrieval 품질은 보장하지 못했다. 기본 전략이 아니라 fallback으로 제한해야 한다.

정확도 측면에서 pointer 방식은 source/runtime 실험에서 명확히 우수했다. Symbol 실험에서는 bundle 확장이 retrieval 문제를 일부 해결했지만 reasoning 오류가 남았다. 따라서 최종 시스템은 pointer retrieval뿐 아니라 deterministic computation, schema validation, test execution feedback을 함께 포함해야 한다.

5 Conclusion

본 보고서는 회사 내부 코드와 데이터를 외부 클라우드 모델로 전송할 수 없는 환경에서 사용할 수 있는 로컬 LLM 기반 코드 어시스턴스 시스템을 제안했다. 이 문제에서 보안은 단순한 부가 조건이 아니라 시스템 구조를 결정하는 핵심 제약이다. 클라우드 기반 코드 어시스턴스는 큰 모델과 강한 tool-use 성능을 사용할 수 있지만, 내부 repository, 운영 로그, tenant 설정, 테스트 출력이 외부 API로 전달될 수 있다. 반대로 로컬 LLM은 데이터 유출 위험을 낮추지만, 모델 크기, 양자화, context window, prompt evaluation latency, long-context retrieval 품질에서 제약을 갖는다. 따라서 보안형 코드 어시스턴스는 “클라우드 모델을 로컬 모델로 바꾸는 것”만으로는 충분하지 않고, 로컬 모델이 처리해야 하는 context 자체를 작고 검증 가능하게 설계해야 한다.

실험 결과는 full-context 방식이 이 목표에 적합하지 않음을 보여준다. Symbol 실험에서 full_repo는 medium corpus를 65k context에 넣지 못했고, 131k context로 늘려도 한 질문에 612.869초가 걸렸다. 이는 전체 repository를 prompt에 넣는 방식이 context overflow뿐 아니라 latency 측면에서도 실제 agent loop에 부적합하다는 뜻이다. Source pointer와 runtime pointer 실험에서도 같은 패턴이 반복되었다. Medium source 실험의 full_source는 log와 JSON에서 context overflow가 발생했고 정확도는 1/3에 그쳤다. Medium runtime 실험의 full_runtime도 log와 JSON에서 overflow가 발생했다. 즉 코드 어시스턴스가 다루는 데이터는 코드 파일만이 아니라 외부 문서, 로그, JSON, 실행 중간 결과까지 포함하므로, 모든 데이터를 raw prompt로 합치는 방식은 확장 가능한 기본 전략이 될 수 없다.

제안 시스템은 이 한계를 pointer 기반 접근 계층으로 해결한다. Repository 내부 코드는 memory://symbol/... pointer로, disk에 존재하는 외부 자료는 source://... pointer로, 실행 중 생성된 tool output은 runtime://... pointer로 관리한다. 이 분리는 데이터의 lifecycle과 접근 방식이 서로 다르기 때문에 필요하다. Code symbol은 AST index와 line range provenance가 필요하고, source file은 manifest와 file resolver가 필요하며, runtime object는 session-local store와 cleanup 정책이 필요하다. 세 계층을 구분하면 각 데이터 유형에 맞는 resolver와 extractor를 붙일 수 있고, LLM에는 전체 원본 대신 compact catalog, 선택된 symbol bundle, 추출된 log line, JSON field 같은 작은 evidence만 전달할 수 있다.

정량 결과도 이 구조를 뒷받침한다. Source pointer 실험에서 medium 기준 pointer_oracle은 prompt token을 120,208에서 286으로 줄였고, wall time도 22.117초에서 2.443초로 줄었다. pointer_select 역시 평균 780 token만 사용하면서 3/3 정확도를 유지했다. Runtime pointer 실험에서도 medium 기준 runtime_pointer_oracle은 prompt token을 131,136에서 311로 줄였고, wall time을 26.270초에서 2.610초로 줄였다. 중요한 점은 token 절감 자체보다 데이터 크기와 LLM context 크기가 분리되었다는 것이다. Runtime JSON이 134,023 characters이든 더 커지든, LLM이 받는 것은 pointer catalog와 extractor output뿐이다. 이 특성은 실제 코드 어시스턴스에서 테스트 로그나 build output이 커질수록 더 중요해진다.

Symbol 실험은 코드 접근 계층에서 두 가지 교훈을 준다. 첫째, 단일 symbol 접근은 매우 효율적이다. Medium corpus에서 symbol_oracle은 평균 199 prompt token과 2.131초만 사용하면서 기본 task 3개를 모두 맞혔다. 둘째, 실제 코드 이해 질문은 단일 함수 본문만으로 충분하지 않을 수 있다. Hard task에서 symbol_select는 대표 symbol URI를 모두 올바르게 선택했지만 최종 정확도는 0/3이었다. 이유는 선택된 함수 안에 답이 완전히 들어 있지 않았기 때문이다. 예외 메시지는 module-level constant에 있고, cache key 계산은 helper 함수가 필요하며, billing 질문은 서로 다른 symbol을 함께 봐야 했다. 이를 보완하기 위해 expand_symbol_context를 사용한 bundle 방식은 기존 symbol_select 대비 hard task에서 prompt token을 33,922에서 587로 줄이고 wall time을 172.170초에서 4.588초로 줄였다. 다만 정확도는 1/3에 머물렀는데, 이는 retrieval이 아니라 문자열 변환과 산술 계산 같은 execution 단계에서 발생한 오류였다.

따라서 최종 시스템은 retrieval만 잘하는 구조로는 부족하다. 코드 어시스턴스의 작업은 관련 파일과 symbol을 찾는 retrieval, 큰 자료에서 필요한 부분을 잘라내는 extraction, 찾은 코드를 실제 입력값에 대해 계산하는 deterministic execution, selector와 patch의 형식을 검사하는 validation으로 나누어야 한다. 이 분리는 오류가 발생하는 위치를 분명히 하고, LLM이 잘하지 못하는 일을 deterministic tool로 대체하기 위해 필요하다. 예를 들어 JSON path 추출은 LLM이 눈으로 긴 JSON을 읽게 하는 것보다 extract_json_field가 수행하는 편이 안정적이고, 문자열 변환 결과는 LLM이 추측하게 하는 것보다 작은 evaluator가 계산하는 편이 정확하다. 또한 selector가 만든 pointer나 tool call은 schema와 allowlist로 검증해야 잘못된 tool 실행이나 잘못된 파일 접근을 막을 수 있다.

Self-Extend 실험은 “context window를 늘리면 문제가 해결된다”는 가정에 한계가 있음을 보여준다. G=2와 G=4 조건은 입력 수용량을 늘렸지만 passkey retrieval과 code assistance benchmark에서 정답을 회수하지 못했다. 특히 code assistance benchmark에서는 off 조건이 prompt too long으로 실패했고, G=2/G=4 조건은 입력을 받았지만 함수 반환값, symbol reference, failing test 원인을 모두 맞히지 못했다. 이는 긴 prompt를 넣을 수 있는 능력과 긴 prompt 안에서 필요한 코드 사실을 정확히 찾아 사용하는 능력이 다르다는 점을 보여준다. 따라서 Self-Extend는 본 시스템의 기본 retrieval 방식이 아니라, pointer 기반으로 context를 줄인 뒤에도 필요할 때 제한적으로 검

증하며 사용하는 fallback이어야 한다.

본 연구의 남은 한계도 명확하다. 실험 corpus는 synthetic Python 중심이므로 실제 repository의 import graph, framework convention, decorator, dynamic dispatch, generated code, build system, test fixture 복잡도를 충분히 반영하지 못한다. 또한 현재 `expand_symbol_context`는 same-file constant와 helper 중심이므로 cross-file dependency를 따라가는 능력이 제한적이다. 마지막으로 bundle에 필요한 코드가 들어와도 LLM이 계산을 틀릴 수 있으므로 deterministic evaluator가 아직 핵심 미완성 요소로 남아 있다.

실제 제품 구현에서는 사용자가 pointer 관리 자체를 의식하지 않도록 만드는 것도 중요하다. `symbol_index`는 workspace open, file save, agent patch 적용 시점에 자동으로 갱신되어야 하며, 사용자는 “index를 다시 만들라”고 직접 지시하지 않아도 되어야 한다. `source_pointer`는 모든 disk file을 훑는 방식이 아니라 project별 allowlist와 file kind 설정으로 제한해야 하고, 사용자가 질문에서 파일 경로를 언급하면 필요한 entry만 즉시 등록하는 lazy 방식을 제공해야 한다. `runtime_pointer`는 tool output 크기와 type에 따라 자동으로 pointer화하되, session TTL, LRU cleanup, 사용자가 보존한 report의 source pointer 승격 정책을 가져야 한다. `self_extend`는 사용자가 직접 attention 설정을 고르는 기능이 아니라, pointer 경로가 실패했을 때 agent가 제한적으로 시도하고 검증 결과를 사용자에게 설명하는 fallback으로 감춰지는 편이 사용성이 좋다.

향후 작업은 이 한계를 직접 줄이는 방향이어야 한다. 첫째, tree-sitter 기반 다중 언어 parser를 붙여 Python 외의 TypeScript, Java, C/C++ 코드에도 같은 symbol pointer 구조를 적용해야 한다. 둘째, import graph와 call graph를 이용해 cross-file symbol bundle을 구성해야 한다. 셋째, 문자열 처리, 산술, 작은 함수 실행, JSON path 평가를 담당하는 deterministic evaluator를 추가해야 한다. 넷째, source/runtime pointer catalog와 selector output에 schema validation, path normalization, fallback extraction을 넣어 tool-call 안정성을 높여야 한다. 다섯째, file watcher, index versioning, source root allowlist, runtime object TTL을 하나의 lifecycle manager로 묶어 pointer가 오래되거나 과도하게 쌓이는 문제를 방지해야 한다. 마지막으로 실제 오픈소스 repository나 내부와 유사한 대형 benchmark에서 반복 실험을 수행해 정확도, latency, token 사용량의 분산을 측정해야 한다.

결론적으로, 보안형 로컬 코드 어시스턴스의 핵심은 더 긴 prompt를 무작정 허용하는 것이 아니라, LLM이 봐야 하는 evidence를 작고 정확하게 만드는 것이다. Pointer 기반 code/source/runtime 접근 계층은 로컬 LLM의 context 제약과 보안 요구를 동시에 만족시키는 실용적인 구조이며, symbol bundle과 deterministic execution을 결합하면 실제 코드 수정 workflow에 필요한 신뢰성을 더 높일 수 있다.